

Autism Screening Risk Prediction

Prepared By Rafa al-faiz

Supervisor : Ms.Raghad Al-Shammari

Introduction

For my final project, I chose the **Autism Screening Risk Prediction** dataset because it is a clear classification problem and it fits the topics covered in the course. The main goal of this project is to build models that can predict the Class/ASD result based on screening question scores and other general information in the dataset.

The target column is Class/ASD, which contains two classes: 0 and 1. The models learn from features such as A1_Score to A10_Score, age, gender, and other related columns, then predict the final class. This project is not a medical diagnosis; it is only a machine learning project based on screening data. Through this project, I practiced the full machine learning process, including data loading, exploration, preparation, model training, evaluation, and comparison.

Dataset Description

The dataset contains **800 rows and 22 columns**. It includes screening question scores from A1_Score to A10_Score, along with general features such as age, gender, ethnicity, jaundice, country of residence, and relation. The target column is Class/ASD, which the models try to predict.

I checked the dataset using `df.head()`, `df.shape`, and `df.isnull().sum()`. The dataset had **no missing values**, so no filling or dropping was needed. I also checked the class balance using `value_counts()`, and the result showed that class 0 had **639 samples** while class 1 had **161 samples**. This means the dataset is imbalanced, so I focused on **F1-score** as the main evaluation metric.

Models Used

In Step 1, I trained three machine learning models: Logistic Regression, Random Forest, and KNN. Before training, I separated the features into X and the target column Class/ASD into y. I also used `pd.get_dummies()` to convert categorical columns into numeric values, then split the data into 80% training and 20% testing using `random_state=42` and `stratify=y`.

The Logistic Regression model achieved the highest F1-score, about **0.67**, so it was the best machine learning model. Random Forest achieved an F1-score of about **0.62**, while KNN had the lowest performance with an F1-score of about **0.26**. Since the dataset is imbalanced, I focused on F1-score more than accuracy when choosing the best model.

Neural Network

In Step 2, I built a neural network using Keras to compare it with the machine learning models from Step 1. The model was created using Sequential, which means the layers are added one after another. I used Dense layers with ReLU activation in the hidden layers because ReLU helps the model learn patterns from the data.

I also added a Dropout layer to reduce overfitting and help the model avoid memorizing the training data too much. Since this is a binary classification problem, I used Sigmoid activation in the output layer because the target has two classes, 0 and 1.

The model was compiled using Adam optimizer and binary crossentropy loss, then trained for 20 epochs. After training, I plotted the training and validation loss and accuracy curves to check the learning process. The neural network achieved Accuracy 0.81, Precision 1.00, Recall 0.06, and F1-score 0.12.

Results & Comparison

I compared all models using the same evaluation metrics: accuracy, precision, recall, and F1-score. Since the dataset is imbalanced, I focused mainly on F1-score instead of accuracy, because F1-score gives a better balance between precision and recall.

Logistic Regression achieved the best overall result because it had the highest F1-score. Random Forest performed close to Logistic Regression, but its F1-score was slightly lower. KNN had the weakest performance because it had low recall and low F1-score, which means it did not detect the positive class well.

The neural network achieved good accuracy and high precision, but its recall was very low. This means that when it predicted class 1, it was usually correct, but it missed many actual class 1 cases. Because of that, its F1-score was low compared to Logistic Regression.

Overall, **Logistic Regression** was the best model in this project because it had the best balance between precision and recall, which is important for an imbalanced dataset.

What I Learned

From this project, I learned how to build a full machine learning pipeline, starting from loading and exploring the dataset to training, evaluating, and comparing models. I also learned that checking class balance is important because imbalanced data can make accuracy misleading.

In this project, F1-score was more useful than accuracy because it balances precision and recall. I also learned that complex models are not always better, since Logistic Regression performed better than the neural network in this case. This helped me understand the importance of choosing the right evaluation metric and comparing models carefully.